

DESAFIO TÉCNICO

Banking as a Service (BaaS) Integração com Gateway

VBA Systems

Ambiente de simulação monitorado

Objetivo

Avaliar a capacidade de construir uma aplicação no mesmo espírito da arquitetura utilizada no dia a dia, integrando um produto BaaS ao gateway de pagamento simulado Lera Box como provedor de processamento.

Escopo central

- Checkout Pix e cartão
- Carteira e extrato
- Saques
- Webhooks
- Tarifas de cartão
- Conciliação por referência externa

Classificação: desafio técnico para avaliação de desenvolvimento e integração.

1. Contexto

O candidato construirá um sistema no estilo Banking as a Service (BaaS). A aplicação será o produto utilizado pelo lojista ou usuário, enquanto o gateway Lera Box será o processador das operações.

- Checkout Pix e cartão
- Carteira
- Saques
- Webhooks
- Tarifas de cartão
- Extrato

Ambiente de simulação

API: <https://api.branchpay.com.br>

Swagger: <https://api.branchpay.com.br/doscs>

2. Stack obrigatória

Backend

Tecnologia	Uso esperado
TypeScript + NestJS	API da aplicação BaaS
TypeORM	Persistência
class-validator / class-transformer	DTOs e validação de entrada
Swagger (@nestjs/swagger)	Documentação da API do candidato
Middleware Nest	Logging, correlation id e apoio a autenticação
MySQL	Banco próprio da aplicação BaaS

Frontend

Tecnologia	Uso esperado
React + Vite	Interface do lojista e checkout

A aplicação BaaS deve possuir banco próprio. É proibido acessar diretamente o banco do gateway; toda integração deve ocorrer pelas APIs HTTP disponibilizadas.

3. Escopo funcional obrigatório

3.1 Conta no gateway

- Realizar cadastro público no gateway por POST /api/users, para PF ou PJ, usando e-mail e telefone reais.
- Receber por e-mail: documento, senha, CódigoCliente e ChaveLoja.
- Realizar login em POST /api/auth/login.
- Armazenar o Bearer token com segurança no backend BaaS.
- Nunca expor a senha do gateway no frontend.

3.2 Checkout - link de pagamento

Criar no produto BaaS a noção de link ou sessão de checkout com identificador próprio e referência conciliável no gateway.

- Pix: chamar POST /api/payments/pix e exibir qrCodeBase64 e/ou EMV ao pagador.
- Cartão: consultar GET /api/fees, escolher parcelas e enviar installments + feePercent corretos em POST /api/payments/card.
- Usar externalReference alinhado ao gateway para conciliação.

3.3 Transações e filtros

Listar as transações do lojista com, no mínimo, os seguintes filtros:

Filtro da UI	Mapeamento sugerido
Sucesso	APPROVED
Falha	DENIED
Expirado	EXPIRED - estado do link/checkout e/ou gateway
Cancelado	CANCELLED - estado do link/checkout e/ou gateway

Utilizar também o extrato do gateway: GET /api/wallet/transactions?status=&type=&limit=.

3.4 Carteira

- Exibir saldo por GET /api/wallet.
- Apresentar extrato consolidado na interface.

3.5 Saque

- Solicitar saque para outra conta ou chave Pix por POST /api/withdrawals.
- Consultar o status por GET /api/withdrawals/:id.

3.6 Webhooks

- Cadastrar no gateway URLs da API BaaS para PAYMENT_PIX, PAYMENT_CARD e WITHDRAWAL.
- Implementar endpoints receptores autenticáveis e validáveis.
- Validar X-Lera-Box-Signature quando houver secret.
- Tratar o retorno assíncrono definitivo de Pix e cartão por webhook.
- Atualizar o status local do pedido e do link de checkout.

3.7 Taxas

- Consultar GET /api/fees e, quando aplicável, ?brand= antes de montar o link de cartão.
- Persistir e exibir a taxa aplicada no momento da criação do link.
- Rejeitar ou evitar o envio de feePercent divergente; o gateway também fará essa validação.

Todos os valores monetários recebidos ou enviados ao gateway devem ser tratados em centavos.

4. Extras - diferenciais

- Docker para API, frontend e MySQL, com docker compose.
- Domínio e deploy em VPS simples com HTTPS.
- Envio do link de pagamento por e-mail e/ou WhatsApp.
- Comprovante de pagamento em PDF ou página imprimível após o sucesso.

5. Contrato resumido do gateway

Base: <https://api.branchpay.com.br/api>

Swagger: <https://api.branchpay.com.br/doscs>

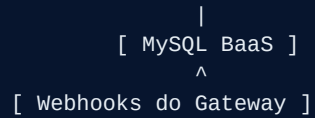
Autenticação protegida: Authorization: Bearer <access_token>

Método	Rota	Auth	Observação
POST	/users	Pública	Cadastro; resposta somente com mensagem
POST	/auth/login	Pública	Token + CódigoCliente + ChaveLoja
POST	/auth/reset-password	Pública	Nova senha por e-mail
GET	/users/me	Bearer	Perfil
GET	/fees	Pública	Taxas Visa/Master/Elo de 1 a 21x
GET	/wallet	Bearer	Saldo em centavos
GET	/wallet/transactions	Bearer	Extrato e filtros
POST	/payments/pix	Bearer	Retorna EMV, QR e txid
POST	/payments/card	Bearer	Exige feePercent da tabela
GET	/payments/:id	Bearer	Consulta do próprio usuário
POST	/withdrawals	Bearer	Solicitação de saque
GET	/withdrawals/:id	Bearer	Consulta do saque
POST/GET/D ELETE	/webhooks	Bearer	Cadastro e gestão de callbacks

Isolamento: cada token deve enxergar apenas os dados da própria conta, incluindo carteira, transações e webhooks.

6. Arquitetura sugerida

[React / Vite] -> [NestJS BaaS API] -> [Gateway Lera Box]



Responsabilidades típicas no BaaS

- Users e sessão do lojista, que podem ser distintos do usuário do gateway.
- Checkout links e orders, com status local e identificadores do gateway.
- Webhook events para registrar payloads recebidos.
- Jobs ou handlers responsáveis por atualizar pedidos após webhooks.

Modelo mínimo de dados sugerido

Entidade	Finalidade
users	Usuários da aplicação BaaS
gateway_accounts	Vínculo seguro com a conta e credenciais do gateway
checkout_links	Links, valores, método, taxa, expiração e referência
orders	Pedido e estado de processamento
transactions	Espelho/conciliação de transações relevantes
withdrawals	Solicitações e status dos saques
webhook_events	Auditoria, idempotência e processamento dos callbacks

Recomenda-se proteger tokens e segredos no backend, usar correlação de requisições, processar webhooks com idempotência e nunca confiar em atualizações de status vindas do frontend.

7. Critérios de avaliação

#	Critério
1	Organização NestJS: módulos, DTOs, validators e Swagger
2	Uso correto de TypeORM e MySQL
3	Middleware e guards coerentes
4	Integração fiel ao gateway: auth, fees, webhooks e isolamento
5	UX mínima no React: link, saldo, filtros, saque e webhooks
6	Qualidade do README: setup, variáveis e fluxos
7	Diferenciais: Docker, VPS, envio de link e comprovante

8. Entrega

- Repositório contendo backend e frontend.
- README com instruções de setup local.
- URL do Swagger da aplicação BaaS.
- URL pública e/ou Docker, quando disponível.
- Credenciais de demonstração da aplicação do candidato; evitar compartilhar senha do e-mail usado no gateway.

Prazo e canal de entrega serão informados pelo recrutamento ou pelo time técnico.

9. Observações do sandbox

- Os status de Pix, cartão e saque podem ser simulados com aprovação ou negação aleatória.
- A interface e os webhooks devem tratar corretamente sucesso e falha.
- CPF/CNPJ podem ser fictícios no gateway.
- E-mail e telefone usados no cadastro devem ser válidos.
- Em caso de dúvida de contrato, priorizar o Swagger do gateway.

Boa integração!

Checklist consolidado

Área	Itens de verificação
Backend e persistência	NestJS + TypeScript; TypeORM + MySQL; DTOs e validators; Swagger; middleware/guards; banco próprio
Integrações	Cadastro e login; token seguro; fees; Pix; cartão; wallet; extrato; withdrawals; webhooks; externalReference
Frontend	React + Vite; criação de link; QR Pix; parcelas e taxas; saldo; filtros; saque; configuração de webhook
Segurança	Sem senha do gateway no frontend; sem segredos no Git; assinatura de webhook; isolamento por conta
Qualidade	README reproduzível; envs; erros tratados; valores em centavos; código organizado; logs e idempotência
Diferenciais	Docker compose; deploy HTTPS; envio de link; comprovante em PDF ou página imprimível

Resultado esperado

Uma aplicação BaaS funcional, com integração HTTP fiel ao gateway, persistência própria, experiência mínima para o lojista e tratamento confiável de eventos assíncronos.